Lab Manual 2: Implementation of the Fundamental Programming Structure in Java

Objective

This lab aims to introduce students to the fundamental programming structure in Java. Students will:

✅ Install and set up Java in **Visual Studio Code**
✅ Learn Java **syntax and concepts** (Variables, Data Types, Operators, Control Statements, Loops)
✅ Write and execute basic Java programs

1. Setting Up Java in Visual Studio Code

Step 1: Install Java JDK

1. Download and install **Java JDK (Java Development Kit)** from: https://www.oracle.com/java/technologies/javase-downloads.html
2. Check the installation by running in cmd or power shell:

Java -version

Step 2: Install Visual Studio Code

1. Download and install **VS Code** from: https://code.visualstudio.com/

Step 3: Install Java Extensions in VS Code

1. Open VS Code and go to **Extensions (Ctrl+Shift+X)**
2. Search for **"Extension Pack for Java"** and install it

Step 4: Create a Java Project in VS Code

1. Open VS Code and create a new folder **(JavaPrograms)**
2. Inside the folder, create a new Java file **(MyFirstProgram.java) or** any name you want to give your project.
3. Write the following basic Java program:

```
// This is a simple Java program that prints "Hello, Java!"
public class MyFirstProgram {
    public static void main(String[] args) {
        System.out.println("Hello, Java!"); // Print message to console
    }
}
```

Arithmetic operations are essential in any programming language as they allow us to perform mathematical calculations. Java provides several built-in operators to handle arithmetic operations.

Types of Arithmetic Operators in Java

1. **Addition (+)** → Adds two numbers.
2. **Subtraction (-)** → Subtracts one number from another.
3. **Multiplication (*)** → Multiplies two numbers.
4. **Division (/)** → Divides one number by another (returns quotient).
5. **Modulus (%)** → Finds the remainder when one number is divided by another.

Key Points to Remember:

- **Division (/) in Java**:

    a. If both numbers are **integers**, the result will be an **integer** (decimal part is removed).
    b. If at least one number is a **floating point number**, the result will be **floating point**.

- **Modulus (%) is often used**:

    a. To check if a number is even or odd (num % 2 == 0 means even).

Why do we use Arithmetic Operations?

- Used in **mathematical calculations** in software applications.
- Essential for **scientific computing, banking software, gaming, and e-commerce** applications.
- Helps in solving real-world problems like **loan calculations, temperature conversions, and business analytics**.

✧ While Loop in Java

A **while loop** is used when we **do not** know how many times the loop will run beforehand. It executes **until** a specified condition becomes false.

Syntax Explanation:

```
while(condition) {
   // Code to execute }
```

- The **condition is checked first**.
- If the condition is true, the block inside {} executes.
- After each iteration, the condition is **checked again**.
- If the condition is false, the loop terminates.

When do we use While Loop?

- Taking user input **until** they enter a valid value.
- Running a program **until** a user selects an exit option.
- Simulating real-world scenarios like **game loops, timers, and waiting mechanisms**.

✧ For Loop in Java

A **for loop** is used when the number of iterations is known beforehand. It executes a block of code multiple times in a controlled manner.

Syntax Explanation:

```
for(initialization; condition; increment/decrement) {
    // Code to execute
}
```

- **Initialization** → The loop variable is initialized once before the loop starts.
- **Condition** → The loop runs **as long as** this condition is true.
- **Increment/Decrement** → Updates the loop variable after each iteration.

Use Cases of For Loop:

- Printing numbers from 1 to N.
- Iterating over arrays and lists.
- Generating patterns (like stars * in a triangle format).
- Performing repeated calculations, such as calculating factorials.

Why do we use For Loop?

- **Efficiency** → It avoids writing repetitive code manually.
- **Control** → The number of iterations is known in advance.
- **Optimization** → Helps in reducing runtime complexity by efficiently handling repeated tasks.

Key Differences Between For Loop and While Loop:

Feature	For Loop	While Loop
Used When	Number of iterations is **known**	Number of iterations is **unknown**
Initialization	Inside loop header	Outside loop
Best for	Iterating over arrays, lists	Running an operation until a condition is met

✧ If-Else Condition in Java

An **if-else statement** is used to make **decisions** in a program. Based on a condition, different parts of the program execute.

How Does If-Else Work?

1. The condition inside the if statement is **checked**.
2. If the condition is **true**, the code inside {} executes.
3. If the condition is **false**, the code inside the else block executes.

Syntax Explanation:

```
if(condition) {
    // Code runs if condition is true
```

```
} else {
    // Code runs if condition is false
}
```

Types of If-Else Statements:

1. **Simple If Statement**

    1. Executes a block of code **only if** a condition is true.

```
if(age >= 18) {
    System.out.println("You are eligible to vote.");
}
```

2. **If-Else Statement**

- If the condition is false, an alternate block of code runs.

```
if(age >= 18) {
    System.out.println("You can vote.");
} else {
    System.out.println("You cannot vote.");
}
```

3. **Nested If-Else**

- An if-else block inside another if-else block.

```
if (age >= 18) {
    if (citizenship.equals("Pakistani")) {
        System.out.println("You can vote in Pakistan.");
    } else {
        System.out.println("You are not eligible to vote in Pakistan.");
    }
} else {
    System.out.println("You are too young to vote.");
}
```

4. **Else-If Ladder**

- Used when multiple conditions need to be checked.

```
if (marks >= 90) {
    System.out.println("Grade: A");
} else if (marks >= 80) {
    System.out.println("Grade: B");
} else if (marks >= 70) {
    System.out.println("Grade: C");
} else {
    System.out.println("Fail");
}
```

Why do we use If-Else?

- Helps in making **decisions** in programs.
- Used in **login authentication, grading systems, user validation, and form submissions**.
- Ensures programs react **dynamically** based on user input or system conditions.

2- Basic Java Programs

Program 1: User Input and Arithmetic Operations

```
import java.util.Scanner; // Import Scanner for user input

public class ArithmeticOperations {
   public static void main(String[] args) {
      Scanner input = new Scanner(System.in); // Create
Scanner object

      // Ask user to enter two numbers
      System.out.print("Enter first number: ");
      int num1 = input.nextInt(); // Read first number

      System.out.print("Enter second number: ");
      int num2 = input.nextInt(); // Read second number

      // Perform arithmetic operations
      int sum = num1 + num2;
      int difference = num1 - num2;
      int product = num1 * num2;
      int quotient = num1 / num2;
      int remainder = num1 % num2;

      // Print results
      System.out.println("Sum: " + sum);
      System.out.println("Difference: " + difference);
      System.out.println("Product: " + product);
      System.out.println("Quotient: " + quotient);
      System.out.println("Remainder: " + remainder);

      input.close(); // Close Scanner
   }
}
```

Progr

```
public class WhileLoopExample {
   public static void main(String[] args) {
      int i = 1; // Initialize counter

      // Print numbers from 1 to 10
      while (i <= 10) {
         System.out.println(i);
         i++; // Increment counter
```

Program 3: If-Else Condition (Check Even or Odd)

```
import java.util.Scanner;

public class EvenOddChecker {
   public static void main(String[] args) {
      Scanner input = new Scanner(System.in); // Create Scanner object

      System.out.print("Enter a number: ");
      int num = input.nextInt(); // Read number from user

      // Check if number is even or odd
      if (num % 2 == 0) {
         System.out.println(num + " is an Even number.");
      } else {
         System.out.println(num + " is an Odd number.");
      }

      input.close(); // Close Scanner
   }
}
```

Program 4: For Loop Example (Multiplication Table)

```
import java.util.Scanner;

public class MultiplicationTable {
   public static void main(String[] args) {
      Scanner input = new Scanner(System.in); // Create Scanner object

      System.out.print("Enter a number: ");
```

Lab 2 Tasks

Lab 2 Tasks

Lab Task 1:
Create a **simple quiz game** that:

- Asks **3 multiple-choice questions**.

- Gives the user **4 options** for each question.
- If the user selects the correct answer, they get **1 point**.
- At the end, display the **final score** out of 3.

Write a Java program to **implement this mini quiz game** using **if-else statements and a loop**.

Lab Task 2:

A university wants to develop a **Grade Calculator**.

- The program should ask the student for **5 subject marks** (out of 100).
- It should calculate the **average** of the marks.
- Based on the average, assign a **grade** as follows:

    a. **90-100** → Grade **A**
    b. **80-89** → Grade **B**
    c. **70-79** → Grade **C**
    d. **60-69** → Grade **D**
    e. **Below 60** → Grade **F**

Write a Java program to **input 5 subject marks, calculate the average, and display the grade.**

Lab Task 3:

A bank ATM system requires a **4-digit PIN** to access the account.

- The user is given **3 attempts** to enter the correct PIN.
- If the correct PIN is entered, print **"Access Granted"**.
- If the wrong PIN is entered **3 times**, print **"Account Locked"**.

Write a Java program using a **loop** to allow the user **three attempts to enter the correct PIN**.

Lab Task 4:

Write a Java program that asks the user for a number and tells whether it is even or odd.

Lab Task 5:

Write a Java program that takes a number as input and prints its multiplication table up to 100

Lab Task 6:

Ask the user to enter their marks and determine if they have passed (if marks are 50 or more) or failed.

Lab Task 7:

Ask the user to enter a single character and determine if it is a vowel (a, e, i, o, u) or a consonant